
SSH Negotiation Failures

The Problem: The "Security Mismatch"

Modern Linux (Ubuntu 22.04+) has deprecated **SHA-1** and **RSA-1024** because they are mathematically vulnerable. Older Cisco IOS versions still rely on these for their "Key Exchange" (Kex) and "Host Key" algorithms.

The Error: `no matching key exchange method found`

Phase 1: The "Manual Override" (Option 1)

Use this if you are on a jump host you don't own, or if you only need to connect once to fix a config. You are manually telling the SSH client to "lower its shield" for one session.

Bash

```
ssh -o KexAlgorithms=+diffie-hellman-group14-sha1 -o HostKeyAlgorithms=+ssh-rsa  
cisco@10.199.199.5
```

Phase 2: The "Permanent Fix" (Option 2 - Lab Standard)

This is what you just did. It creates a "Policy" on your Ubuntu server that automatically uses the correct settings for your lab subnet.

1. Open (or create) the config file:

Bash

```
nano ~/.ssh/config
```

2. Add the following block:

Plaintext

```
# Apply these settings to all devices in the 10.199.199.x range  
Host 10.199.199.*  
    KexAlgorithms +diffie-hellman-group14-sha1,diffie-hellman-group-exchange-sha1  
    HostKeyAlgorithms +ssh-rsa  
    PubkeyAcceptedAlgorithms +ssh-rsa
```

3. Set correct permissions:

(Linux is strict; if this file is too "public," it will ignore it.)

Bash

```
chmod 600 ~/.ssh/config
```

🔧 Phase 3: The "Specialist's Fix" (The Proper Way)

Instead of making your server **less** secure, make your switch **more** secure. If the switch hardware supports it, do this on the Cisco CLI:

Code snippet

```
conf t
# 1. Generate a modern, 2048-bit key
crypto key generate rsa general-keys modulus 2048

# 2. Tell the switch to use modern algorithms
ip ssh server algorithm kex ecdh-sha2-nistp256
ip ssh server algorithm encryption aes256-ctr
end
```

🌐 The "Real World" Perspective

In a **Banking Institution** or a high-security environment, the rules are different:

- **Audit Compliance:** If an auditor sees `diffie-hellman-group1-sha1` or `RSA-1024` in your network, they will flag it as a **High Risk**.
 - **The Workflow:** In the real world, we do **Phase 3** first. We upgrade the firmware and the keys on the switches to meet modern standards.
 - **The Exception:** You only use **Option 2** when you are managing "Legacy" gear (very old switches) that cannot be upgraded but are still critical to the business.
 - **Netmiko Note:** When you start writing your Netmiko scripts, Netmiko actually has a way to handle this automatically in the code using the `SSH_CONFIG` parameter, but having it fixed at the OS level (your `~/.ssh/config`) is much cleaner.
-